

CREATION D'UNE APPLICATION JAVA :
APPLICATION DE GESTION DE FACTURES
D'ELECTRICITE : EDI ENERGIE

DOCUMENT TECHNIQUE



edi energie

L'ENERGIE QUI VOUS ECLAIRE

EDI ENERGIE

PARTICIPANTS :

- LEUDJEU Eloisa
- TAKAM Duvarelle
- CHERTY Yassine

CLASSE : CPI1-
GROUPE 3IL

ANNEE : 14/3/2026-
29/05/2026

Sous la supervision
de :

Mr Beverini

SOMMAIRE

- I. INTRODUCTION
- II. ARCHITECTURE GENERALE
- III. FLUX PRINCIPAL DE L'APPLICATION (classe main)

INTRODUCTION

Ce document présente l'architecture technique et le fonctionnement interne de l'application de gestion de facture < **EDI ENERGIE** >. Cette application permet à un utilisateur de garder ses factures et renvoyer le montant à payer de sa consommation (en kilowatt) chaque mois.

I. ARCHITECTURE GENERALE

Cette application repose sur un code composé de 08 classes principales à savoir :

1. La classe CLIENT

Cette classe représente les données entrées par un client dans l'application. Elle affiche les informations du client (son nom, son Id, son adresse) à travers la méthode **afficher**. Elle stocke aussi ses informations dans une base de données créée à travers la méthode **sauvegarder**

2. La classe COMPTEUR

Cette classe représente les données d'un compteur électrique associé à ce client. Elle permet à l'utilisateur d'entrer son numéro du compteur qui devra être uniquement de 14 chiffres (la taille normale du numéro de compteur) une vérification qu'on a eu à mettre dans le setter de la propriété associé au numéro du compteur, sa consommation (en KWh) qui est un

nombre positif vérifié dans un setter associé. Elle récupère aussi toutes ces valeurs grâce aux getters tout en les stockant dans la base de données à partir de la requête SQL associée par la méthode **sauvegarder**.

3. La classe **FACTURE**

La classe Facture crée une facture électrique. Elle calcule son coût selon la consommation du client en KWh sachant que le kilowatt est placé à 15 centimes (prix en euro) à travers la méthode **calculermontant**, et affiche les détails de la facture à l'écran par la méthode **afficher**. Elle génère la facture en PDF à travers la méthode **generer** et sauvegarde toutes les factures dans la base de données grâce à une requêtes SQL.

4. La classe **ConnexionDB**

Cette classe sert à connecter l'application à la base de données. Elle est utilisée par toutes les autres classes.

5. La classe **ConnexionGU**

Cette classe correspond à l'interface de connexion de l'application. Elle crée une fenêtre divisée en deux sections dont la première section est celle de la **connexion** créée par la méthode **buildPanelConnexion** et comportant la création des champs (ID de connexion et mot de passe) et un bouton «**se connecter**» créé par la méthode **builonglets** pour donner l'accès à l'interface principale. Et la deuxième portant le nom de **première connexion** créée par **buildPanelInscription** qui donne le droit à l'utilisateur de créer un compte et elle comporte la création des champs (nom, Id, adresse et mot de passe) et un bouton << **créer un compte**>> créé par la méthode **builonglets** également. Cette classe prend en compte la vérification de l'ID du client et de son mot de passe enregistrés dans la base de données lorsque l'utilisateur essaye de se connecter à son compte par la méthode **tenterconnexion**. Elle vérifie aussi le dernier ID utiliser pour le remplir automatiquement à la prochaine ouverture.

6. La classe Id

Cette classe génère automatiquement des nombres auto-incrémentés considérés comme des identifiants à travers la méthode `getandIncrement`.

Exemple : Le premier utilisateur inscrit reçoit l'ID 1000, le second reçoit l'ID 1001 et ainsi de suite.

7. La classe InitDB

Elle a créé la base de donnée de l'application qui comporte 4 tables à savoir : clients, compteurs, factures, utilisateurs.

II. Flux principal de notre application

La classe `MainGU` est la classe principale de notre projet car elle crée l'interface principale de notre application. Elle ouvre une fenêtre divisée en 4 sections soit :

- La première section comporte le titre de l'application et du logo ainsi que le bouton `<<imprimer>>` et `<< exporter PDF>>` tout ceci crée dans le constructeur de la classe et la méthode `buildCenter`. Les fichiers sont importés en PDF par la méthode `exporterpdf` et imprimer par la méthode `imprimerRecu`.
- La deuxième section gère l'affichage des informations du client connecté à travers la méthode `buildficheclient` e, elle crée une nouvelle facture grâce au bouton `+Nouvelle facture` générer la classe `border` dans la méthode `buildsidebar` et quitte l'application par le bouton `<< Quitter>>` crée par
- La troisième section gère l'aperçu du reçu par les méthodes `recuvide` (aide à la sélection de la facture), `afficherRecu` (affiche le contenu

de la facture), et `buildrecupanel` (qui s'occupe du titre de la section) tout en modifiant le contenu de chaque facture par la méthode `chargerfactureclients`.

- Et la dernière section qui liste les factures du client.